

Sistema de Planeamento de Horários

Carolina Cruz n.º 50475
Constança Costa n.º 50541

Orientador: Paulo Pereira

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Sistema de Planeamento de Horários

50475 Carolina Cruz

50541 Constança Costa

Orientador: Paulo Pereira

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

Resumo

O presente relatório descreve o desenvolvimento final do projeto *Sistema de Planeamento de Horários*, realizado no âmbito da unidade curricular de Projeto e Seminário da Licenciatura em Engenharia Informática e de Computadores.

O projeto surgiu da necessidade de simplificar a organização de horários entre professores e alunos, num contexto em que esta tarefa é frequentemente realizada de forma manual, demorada e sujeita a reajustes constantes. A solução proposta tem como objetivo apoiar esse processo através do registo de utilizadores, da recolha de disponibilidades, da definição de restrições e da construção de propostas de horário de forma estruturada.

A solução desenvolvida integra uma aplicação móvel Android, um backend com API REST implementado em Ktor e persistência central em PostgreSQL. Entre as funcionalidades implementadas destacam-se a autenticação de utilizadores, a associação entre alunos e professores, a definição de disponibilidades com múltiplos intervalos por dia, a configuração de restrições do professor e do aluno, a criação automática de propostas de horário e a persistência de aulas concretas com datas reais.

Para além da criação do horário semanal, o sistema suporta recorrência de aulas, consulta de histórico, marcação de presenças, cancelamento e remarcação de aulas, envio de notificações por email aos alunos e integração do horário aceite com o Google Calendar do professor. Foram ainda realizados testes unitários e de integração sobre componentes do frontend e do backend, permitindo validar os principais fluxos funcionais e regras de negócio.

Os resultados obtidos demonstram a viabilidade técnica e prática da solução, bem como a sua adequação a cenários reais de utilização. O sistema final constitui uma base consistente para evolução futura, nomeadamente ao nível da gestão de salas e da interpretação automática de disponibilidades a partir de texto, imagem ou áudio.

Abstract

This report presents the final development of the project *Scheduling Planning System*, carried out within the Project and Seminar course of the Bachelor's Degree in Computer Science and Computer Engineering.

The project emerged from the need to simplify schedule management between teachers and students, in a context where this task is often performed manually, is time-consuming, and requires frequent adjustments. The proposed solution supports this process through user registration, availability collection, restriction management, and structured schedule generation.

The developed solution integrates an Android mobile application, a backend with a REST API implemented in Ktor, and central persistence in PostgreSQL. The implemented features include user authentication, teacher-student association, availability definition with multiple time blocks per day, teacher and student restriction management, automatic schedule proposal generation, and persistence of concrete lessons with real calendar dates.

In addition to weekly schedule generation, the system supports lesson recurrence, history consultation, attendance registration, lesson cancellation and rescheduling, email notifications to students, and integration of accepted schedules with the teacher's Google Calendar. Unit and integration tests were also carried out for both frontend and backend components, validating the main functional flows and business rules.

The obtained results demonstrate the technical and practical feasibility of the solution, as well as its suitability for real-world usage scenarios. The final system provides a consistent basis for future evolution, namely in classroom management and automatic availability interpretation from text, image, or audio.

Utilização de Inteligência Artificial

No desenvolvimento deste projeto foram utilizadas ferramentas de inteligência artificial como apoio complementar em diferentes fases do trabalho, nomeadamente no esclarecimento de dúvidas, no apoio à organização da documentação, na revisão de texto e na avaliação de possíveis ferramentas e abordagens a utilizar.

Em particular, foram utilizados o ChatGPT, o ambiente Codex e o Claude, como suporte em tarefas como:

- esclarecimento de dúvidas sobre implementação em Kotlin, Jetpack Compose, Ktor e integração entre frontend e backend;
- revisão e reformulação de texto para melhorar a clareza do relatório;
- apoio à identificação de incoerências entre o estado atual da aplicação e a documentação produzida;
- apoio exploratório na análise de possíveis abordagens para evolução funcional do sistema.

Para além disso, no âmbito de uma linha exploratória do projeto, foram também analisadas possibilidades de utilização de modelos locais e multimodais para interpretação de disponibilidades a partir de texto, imagem e, futuramente, áudio. Esta componente foi desenvolvida como estudo complementar e não como funcionalidade concluída da versão final.

A utilização destas ferramentas teve sempre um papel de apoio. As decisões de projeto, a implementação final, a validação do comportamento da aplicação e a redação final do relatório permaneceram sob responsabilidade das autoras, tendo sido sempre efetuada revisão humana sobre os conteúdos produzidos com o apoio de inteligência artificial.

Índice

1	Introdução	1
1.1	Motivação e Objetivos	1
1.2	Visão Geral da Solução	2
1.3	Organização do Documento	3
2	Enquadramento	5
2.1	Problema e Contexto de Aplicação	5
2.2	Trabalho Relacionado e Soluções Semelhantes	6
2.3	Justificação da Solução Proposta	6
3	Arquitetura da Solução	9
3.1	Visão Global da Arquitetura	9
3.2	Arquitetura do Frontend	10
3.3	Arquitetura do Backend	10
3.4	Persistência de Dados	11
3.5	Comunicação entre Frontend e Backend	12
3.6	Considerações sobre a Arquitetura Atual	13
4	Implementação do Sistema	15
4.1	Organização Funcional da Aplicação	15
4.2	Gestão de Utilizadores e Perfis	16
4.3	Gestão de Disponibilidades	16
4.4	Gestão de Restrições	17
4.5	Associação entre Alunos e Professores	17
4.6	Criação de Propostas de Horário	18
4.7	Visualização do Horário Criado	18
4.8	Gestão Dinâmica de Aulas e Presenças	19
4.9	Integração Progressiva com o Backend	19
5	Backend e API	21
5.1	Escolha Tecnológica	21

5.2	Estrutura do Backend	21
5.3	Modelo de Persistência no Backend	22
5.4	API REST Implementada	23
5.5	Criação de Horários no Backend	24
5.6	Serviço de Notificações por Email	25
5.7	Validação com Postman	25
5.8	Integração com a Aplicação Android	26
5.9	Síntese do Estado Atual do Backend	26
6	Testes e Validação	29
6.1	Infraestrutura e Testes Automatizados no Backend	29
6.2	Validação do Backend com Postman	30
6.3	Testes Funcionais da Aplicação Android	31
6.4	Validação da Criação de Horários	32
6.5	Integração e Validação Progressiva	33
6.6	Síntese dos Resultados Obtidos	33
7	Estudo Exploratório com Large Language Models (LLMs) para Interpretação de Disponibilidades	35
7.1	Modelos e Ferramentas Avaliados	35
7.2	Resultados Obtidos com OCR e Multimodalidade	36
7.2.1	Avaliação do GLM-OCR (q8_0)	36
7.2.2	Avaliação do Qwen2.5-VL (3b)	36
7.3	Estado Atual da Componente de Áudio	37
7.4	Proposta de Integração no Projeto	37
7.5	Síntese do Estudo	38
8	Estado Final do Sistema e Avaliação	39
8.1	Funcionalidades Concluídas e Avaliação de Progresso	39
8.2	Limitações Reais e Trabalho Futuro	40
9	Conclusões	41
	Referências	43

Capítulo 1

Introdução

A organização de horários entre professores e alunos é uma tarefa que, em muitos contextos, continua a ser realizada de forma manual, recorrendo a trocas sucessivas de mensagens, folhas e imagens ou registos informais. Este processo tende a ser demorado, pouco flexível e propenso a erros, especialmente quando existem múltiplos alunos, disponibilidades distintas e restrições específicas que precisam de ser respeitadas. A dificuldade aumenta quando se pretende encontrar uma distribuição equilibrada de sessões, compatível com a disponibilidade do professor e com as preferências ou limitações dos alunos.

A motivação para este projeto surgiu de uma experiência concreta das autoras no contexto de um centro de estudos, onde ambas desempenham funções de explicadoras. Neste contexto, a gestão dos horários dos alunos era realizada manualmente e exigia reajustes frequentes sempre que entrava um novo aluno ou surgiam alterações de disponibilidade. Na prática, isso implicava refazer sucessivamente os horários já definidos, tentar encontrar novos encaixes e, muitas vezes, lidar com a dificuldade de conciliar todos os alunos de forma equilibrada e sustentável. Esta experiência permitiu identificar de forma clara a necessidade de uma solução que apoiasse este processo de forma mais eficiente e estruturada.

Neste contexto, surgiu a proposta do projeto *Sistema de Planeamento de Horários*, cujo objetivo consiste no desenvolvimento de uma solução capaz de apoiar a gestão de horários entre professores e alunos. A solução permite recolher disponibilidades, definir restrições de agendamento e construir propostas de horário de forma estruturada, reduzindo o esforço manual associado a este processo e tornando a gestão de sessões mais clara e eficiente.

1.1 Motivação e Objetivos

A motivação principal para o desenvolvimento deste projeto prende-se com a necessidade de criar uma ferramenta capaz de simplificar a coordenação de horários em cenários onde um professor trabalha com vários alunos e necessita de conciliar diferentes disponibilidades ao longo da semana. Em vez de depender exclusivamente de planeamento manual, a solução proposta procura automatizar parte desse trabalho, permitindo que os dados relevantes sejam

recolhidos de forma consistente e utilizados para construir propostas de horário com base em regras bem definidas.

O objetivo central do projeto consiste, assim, em disponibilizar um sistema que permita:

- registar professores e alunos;
- associar alunos a um professor;
- recolher as disponibilidades semanais de professores e alunos;
- definir restrições relevantes para o agendamento;
- construir propostas de horário;
- persistir aulas concretas com datas reais;
- apoiar a gestão operacional das aulas, incluindo consulta, remarcação, cancelamento e marcação de presenças;
- apresentar o horário resultante de forma legível, organizada e acessível.

Numa perspetiva mais ampla, o projeto pretende também servir como base para a evolução de funcionalidades futuras, nomeadamente, ao nível da gestão de salas e do suporte à interpretação automática de disponibilidades a partir de texto, imagem ou áudio.

1.2 Visão Geral da Solução

A solução desenvolvida assenta numa arquitetura composta por uma aplicação Android, responsável pela interação com o utilizador e por um backend com API REST, responsável pela persistência central de dados, pela execução da lógica principal do sistema e pela exposição dos serviços necessários ao frontend. No frontend foi utilizada a linguagem Kotlin com Jetpack Compose para a construção da interface, enquanto no backend foi adotado Ktor para a implementação da API, em conjunto com PostgreSQL para persistência remota.

A aplicação distingue dois perfis principais de utilizador: professor e aluno. Cada um destes perfis dispõe de um conjunto próprio de funcionalidades e ecrãs adaptados ao seu papel no sistema. O professor pode definir a sua disponibilidade semanal, configurar restrições de agendamento, consultar os alunos associados, criar propostas de horário, confirmar essas propostas, persistir aulas concretas com datas reais e gerir o ciclo de vida das aulas já criadas. O aluno pode escolher o professor com quem pretende trabalhar, definir a sua disponibilidade, indicar a sua carga horária semanal pretendida e consultar as aulas que lhe estão atribuídas.

A partir da informação recolhida, o sistema constrói propostas de horário semanal compatíveis com as disponibilidades registadas e com as restrições configuradas. Após a aceitação

da proposta pelo professor, essas sessões são convertidas em aulas concretas com datas reais, recorrência semanal e persistência na base de dados. Sobre essas aulas, o sistema suporta ainda operações como consulta de histórico, marcação de presenças, cancelamento, remarcação e envio de notificações por email aos alunos.

Como complemento prático, o sistema integra também o Google Calendar do professor, permitindo registrar automaticamente no calendário os eventos correspondentes às aulas aceites. Desta forma, a solução aproxima-se de um cenário real de utilização, não se limitando à geração abstrata de horários, mas apoiando também a sua gestão operacional ao longo do tempo.

1.3 Organização do Documento

O presente relatório encontra-se organizado da seguinte forma. No Capítulo 2 é apresentado o enquadramento do problema, a motivação do projeto e o contexto em que a solução se insere. No Capítulo 3 é descrita a organização global do sistema, incluindo os principais componentes e a sua interação. Seguidamente, são apresentados os detalhes de implementação mais relevantes, tanto ao nível do frontend como do backend, bem como os principais aspetos da API, da persistência de dados e da lógica de criação de horários. Posteriormente, é descrita a estratégia de testes e validação aplicada ao sistema. É também apresentado um estudo exploratório sobre a utilização de modelos multimodais para interpretação de disponibilidades a partir de diferentes tipos de input. Por fim, é feito um balanço do estado final do sistema, identificando as funcionalidades concluídas, as limitações atuais, os principais resultados obtidos e as conclusões fundamentais do trabalho realizado.

Capítulo 2

Enquadramento

O planeamento e a organização de horários configuram um desafio clássico de otimização em múltiplos contextos, tais como ambientes educativos, centros de formação e serviços partilhados. A compatibilização de matrizes horárias e restrições operacionais torna-se um processo complexo à medida que o número de participantes escala, exigindo soluções capazes de acomodar preferências individuais, respeitar limites temporais estritos e assegurar uma distribuição homogênea das sessões ao longo da semana.

No âmbito deste projeto, o foco incide sobre a relação bilateral entre docente e discentes. O sistema proposto permite ao professor parametrizar as suas disponibilidades e regras de negócio, enquanto os alunos submetem as suas disponibilidades e definem a sua carga horária semanal pretendida. A partir destes dados, a plataforma cria de forma autónoma propostas de horário compatíveis. Este domínio combina aspetos de gestão de dados, interação utilizador-sistema e lógica de planeamento, tornando o problema simultaneamente prático e tecnicamente relevante.

2.1 Problema e Contexto de Aplicação

O método tradicional e manual de construção de horários assenta em sucessivas trocas de mensagens entre os intervenientes, registos informais e validação empírica de conflitos. Esta abordagem descentralizada perde eficiência quando o volume de alunos aumenta ou perante a volatilidade das agendas. Adicionalmente, a gestão puramente manual dificulta a aplicação rigorosa e sistemática de regras operacionais essenciais, tais como a duração fixa dos blocos, a lotação máxima por sessão e o teto máximo de carga horária diária permitida.

A solução desenvolvida responde a estas limitações através de um ecossistema digital que centraliza a recolha e a estruturação dos dados de agendamento. Em vez de delegar no utilizador o esforço de encontrar combinações viáveis, o sistema processa a informação armazenada para calcular propostas de horário compatíveis com as restrições e disponibilidades registadas. Esta automatização reduz o erro humano por sobreposição, elimina a redundância na comunicação e maximiza a previsibilidade e aproveitamento do tempo útil do professor.

Embora a implementação atual valide este modelo de dados através do fluxo relacional professor-aluno, o contexto de aplicação e a arquitetura desenvolvida são suficientemente abrangentes para permitir a sua extensão. A matriz lógica adotada serve de base para qualquer cenário de acompanhamento individual ou em grupo em que exista a necessidade de coordenar agendas entre um gestor de recursos e múltiplos participantes.

2.2 Trabalho Relacionado e Soluções Semelhantes

O problema abordado insere-se na área de *scheduling* e planeamento de recursos com satisfação de restrições (*Constraint Satisfaction Problems* - CSP). Do ponto de vista conceptual, este domínio exige a modelação de matrizes de disponibilidade, a codificação estruturada de regras de negócio e a execução de algoritmos de emparelhamento (*matching*) entre múltiplos intervenientes. Trata-se de um desafio transversal com forte relevância em sistemas de planeamento académico, gestão de turnos operacionais e alocação dinâmica de recursos organizacionais.

No mercado atual, o ecossistema de soluções digitais de agendamento pode ser dividido em duas categorias principais, ambas apresentando limitações face aos requisitos deste projeto:

- **Sistemas de Agendamento Linear e Individual (e.g., Calendly, Doodle):** Focam-se na marcação de eventos isolados ou fluxos um-para-um, permitindo a seleção de intervalos e o envio automático de confirmações. Contudo, carecem de lógica agregadora para *matching*, falhando na consolidação global de horários semanais recorrentes;
- **Plataformas de Calendário e Agenda Eletrónica (e.g., Google Calendar, Microsoft Outlook):** Funcionam com eficácia como repositórios de eventos e ferramentas de partilha visual. Todavia, a sua arquitetura é essencialmente passiva, não integrando motores algorítmicos capazes de processar dados distribuídos e restrições paramétricas do domínio para inferir uma matriz horária coerente com as disponibilidades e restrições do domínio.

A solução desenvolvida posiciona-se de forma intermédia e especializada face a estas alternativas. Em vez de atuar como um mero repositório estático de eventos, o sistema assume um papel ativo no processo de planeamento. A plataforma combina a gestão relacional de utilizadores, a ingestão descentralizada de disponibilidades e a orquestração de restrições parametrizáveis, estabelecendo uma infraestrutura robusta para a automatização progressiva e inteligente da criação de horários académicos.

2.3 Justificação da Solução Proposta

A escolha de desenvolver uma solução própria justifica-se pela necessidade de responder a requisitos concretos que não são completamente satisfeitos por ferramentas genéricas de ca-

lendário ou por sistemas simples de marcação. O projeto pretende integrar, numa única solução, vários elementos que normalmente surgem dispersos:

- gestão de utilizadores,
- associação entre alunos e professor,
- registo estruturado de disponibilidades,
- configuração de restrições,
- criação automática de propostas de horário,
- persistência de aulas concretas com datas reais,
- gestão de presenças, remarcações e cancelamentos,
- integração com serviços externos, nomeadamente Google Calendar e notificações por email.

Além disso, o desenvolvimento do sistema permite explorar aspetos técnicos relevantes do ponto de vista da engenharia de software, nomeadamente a construção de uma aplicação Android, o desenvolvimento de uma API REST, a persistência remota centralizada de dados e utilização de mecanismos locais de suporte no frontend, e a implementação de lógica de planeamento baseada em restrições. Desta forma, o projeto não se limita a resolver um problema prático, mas constitui também uma oportunidade para consolidar competências técnicas adquiridas ao longo do curso e novas competências relacionadas com a utilização de LLM's e bibliotecas externas.

Capítulo 3

Arquitetura da Solução

A solução desenvolvida assenta numa arquitetura composta por dois blocos principais: uma aplicação móvel Android, responsável pela interação com os utilizadores e um backend com API REST, responsável pela persistência central de dados e pela execução da lógica associada à gestão de informação do sistema. Esta separação permite distribuir responsabilidades de forma clara entre os componentes, simplificando a evolução da solução e favorecendo a manutenção do código.

De forma geral, a aplicação Android permite ao utilizador interagir com o sistema de acordo com o seu perfil, nomeadamente professor ou aluno, enquanto o backend disponibiliza os serviços necessários para registo e consulta de informação. A comunicação entre ambos é realizada através de pedidos HTTP a uma API REST, permitindo que o frontend envie dados estruturados e obtenha respostas consistentes para os vários fluxos funcionais da aplicação.

3.1 Visão Global da Arquitetura

A arquitetura adotada segue uma abordagem em camadas, onde cada componente desempenha uma função específica. No frontend, a interface foi construída com Jetpack Compose, recorrendo a *ViewModels* e repositórios para organizar o estado da aplicação e a lógica de acesso a dados. No backend, a API foi implementada em Ktor, com persistência em PostgreSQL.

Em termos funcionais, a arquitetura pode ser vista como composta pelos seguintes elementos:

- aplicação Android, que apresenta os ecrãs e gere a interação com o utilizador;
- camada de lógica no frontend, responsável por coordenar o estado da interface e os fluxos de utilização;
- API REST no backend, que centraliza as operações de leitura, escrita, autenticação e consulta de dados;

- base de dados PostgreSQL, que armazena a informação persistente do sistema;
- armazenamento local com Room, utilizado como suporte local no dispositivo Android;
- integração com Google Calendar, utilizada para registar no calendário do professor os horários aceites.

Esta organização permite que a aplicação continue a evoluir de forma modular, com fronteiras relativamente claras entre apresentação, lógica de negócio, comunicação remota e persistência.

3.2 Arquitetura do Frontend

O frontend da solução foi desenvolvido em Kotlin para Android, utilizando Jetpack Compose como tecnologia principal para construção da interface. Esta escolha permite uma abordagem declarativa ao desenho dos ecrãs, facilitando a gestão de estados e a criação de componentes reutilizáveis.

A aplicação encontra-se organizada em várias camadas, com destaque para:

- *screens*, responsáveis pela apresentação visual das funcionalidades;
- *ViewModels*, responsáveis pela gestão do estado e coordenação da lógica de apresentação;
- *repositories*, responsáveis pelo acesso a dados locais e remotos;
- entidades locais em Room, que suportam persistência local no dispositivo.

Do ponto de vista funcional, a aplicação distingue dois perfis de utilizador: professor e aluno. Cada perfil possui ecrãs e ações específicas. O professor pode gerir a sua disponibilidade, definir restrições, consultar os alunos associados, criar propostas de horário, confirmar essas propostas, consultar aulas já persistidas, marcar presenças, cancelar ou remarcar sessões e enviar notificações aos alunos. O aluno pode escolher um professor, registar a sua disponibilidade, definir a sua carga horária semanal pretendida, consultar as aulas que lhe estão atribuídas e acompanhar o respetivo estado. Esta separação por papéis foi integrada tanto na navegação da aplicação como na própria lógica dos *ViewModels*.

3.3 Arquitetura do Backend

O backend foi desenvolvido em Kotlin com Ktor, funcionando como servidor de aplicação e ponto central de integração de dados. A API implementada segue o estilo REST e expõe endpoints para operações como:

- criação e consulta de professores e alunos;
- autenticação de utilizadores;
- associação e desassociação de alunos a professores;
- registo, consulta e remoção de disponibilidades;
- registo e consulta de restrições do professor e do aluno;
- criação de propostas de horário;
- geração e persistência de aulas concretas com recorrência;
- consulta de aulas semanais e histórico;
- marcação de presenças;
- cancelamento e remarcação de aulas;
- envio de notificações por email aos alunos.

A estrutura interna do backend encontra-se organizada em componentes como:

- definição de rotas, responsável por mapear pedidos HTTP para operações concretas;
- modelos e DTOs, que representam os dados trocados entre cliente e servidor;
- serviços, que encapsulam partes relevantes da lógica do sistema;
- acesso à base de dados, com apoio de Exposed para modelação e manipulação de tabelas.

A escolha de Ktor permitiu construir um backend leve, relativamente simples de estruturar e bem alinhado com o restante ecossistema Kotlin do projeto. Esta solução revelou-se adequada para suportar o conjunto de operações necessárias e para validar a integração com a aplicação móvel.

3.4 Persistência de Dados

A persistência do sistema está dividida em dois níveis complementares. No backend, os dados principais e o estado real do sistema são centralizados numa base de dados *PostgreSQL*, que funciona como o repositório definitivo da aplicação. No frontend, a utilização de *Room* para persistência local foi reformulada, passou a ter um papel puramente auxiliar, funcionando essencialmente como uma camada de cache para melhorar a fluidez da interface e permitir a consulta rápida de dados já sincronizados.

Na base de dados central, o modelo conceptual foi expandido para abranger não só o planeamento abstrato, mas também a execução e controlo das aulas em tempo real. Assim, são armazenadas as seguintes entidades essenciais ao domínio:

- **Utilizadores e Vínculos:** professores, alunos e a associação explícita entre ambos;
- **Planeamento e Restrições:** disponibilidades horárias de professores e alunos, restrições específicas dos docentes e os limites de horas semanais dos estudantes;
- **Instâncias de Aulas:** aulas concretas calendarizadas com datas reais, associadas ou não a uma série de recorrência;
- **Assiduidade:** a relação entre alunos e cada aula concreta, incluindo o registo individualizado de presenças e faltas.

Esta centralização no PostgreSQL garante que operações críticas como a criação de aulas recorrentes, a validação de conflitos de horários em tempo real e o disparo de notificações por email dependam de um estado único e partilhado por todos os utilizadores.

3.5 Comunicação entre Frontend e Backend

A comunicação entre a aplicação Android e o backend é feita através de pedidos HTTP para a API REST, utilizando o endereço local do servidor durante o desenvolvimento. Os dados trocados entre os componentes são serializados em formato JSON, permitindo uma representação simples e facilmente interpretável por ambas as partes.

Esta comunicação suporta já um conjunto importante de operações, incluindo:

- criação de professores e alunos;
- autenticação de utilizadores;
- consulta de listas de utilizadores;
- associação e desassociação de alunos a professores;
- registo, consulta e remoção de disponibilidades;
- registo e consulta de restrições;
- criação de propostas de horário;
- geração de aulas concretas com datas reais;
- consulta semanal e histórica de aulas;
- marcação de presenças;
- cancelamento e remarcação de aulas;
- envio de notificações e integração com Google Calendar.

A existência desta comunicação remota permite validar o sistema como uma solução efetivamente distribuída, em que a aplicação cliente e o backend colaboram para concretizar os fluxos principais do projeto.

3.6 Considerações sobre a Arquitetura Atual

A arquitetura adotada foi escolhida tendo em conta a necessidade de construir uma solução progressivamente, validando primeiro os fluxos principais e a integração entre componentes, antes de avançar para otimizações mais profundas. A existência simultânea de persistência local e remota reflete a evolução incremental do projeto, permitindo apoiar o desenvolvimento da aplicação móvel ao mesmo tempo que se consolida a API e a base de dados central.

Embora a arquitetura atual já permita demonstrar de forma consistente o funcionamento da solução, existem ainda aspetos a consolidar, nomeadamente o reforço da sincronização entre dados locais e remotos, a evolução da componente de disponibilidade e a melhoria da apresentação dos horários. Ainda assim, a base estrutural da solução encontra-se definida e suficientemente sólida para suportar a utilização do sistema no contexto previsto e a sua evolução futura com novas funcionalidades.

Capítulo 4

Implementação do Sistema

A implementação do sistema foi realizada de forma incremental, começando pela definição dos fluxos principais da aplicação Android e evoluindo depois para a integração com o backend e com a base de dados remota. Esta abordagem permitiu validar gradualmente os principais casos de uso do projeto, garantindo que cada funcionalidade essencial ficava operacional antes da introdução de novos componentes ou maior complexidade.

A solução resultante permite concretizar um conjunto alargado de operações centrais ao domínio do problema, nomeadamente, registo e autenticação de utilizadores, definição de disponibilidades, gestão de restrições, associação entre alunos e professores, criação de propostas de horário, persistência de aulas concretas com datas reais, gestão de presenças, cancelamentos, remarcações e envio de notificações.

4.1 Organização Funcional da Aplicação

A aplicação foi concebida em torno de dois perfis principais de utilizador: professor e aluno. A interface e os fluxos disponíveis são ajustados a cada um destes perfis, permitindo apresentar apenas as operações relevantes para cada caso.

A implementação do frontend procurou manter uma separação clara entre:

- apresentação visual da interface;
- estado da aplicação e lógica de interação;
- acesso a dados locais e remotos.

Desta forma, cada ecrã da aplicação está associado a um conjunto de operações específicas, suportadas por *ViewModels* e repositórios que coordenam a lógica necessária à execução dos respetivos fluxos.

4.2 Gestão de Utilizadores e Perfis

Uma das primeiras preocupações da implementação foi distinguir corretamente os dois tipos de utilizador suportados pelo sistema. Esta separação é essencial porque professores e alunos desempenham papéis diferentes no problema do planeamento de horários.

Na solução implementada:

- o professor pode definir a sua disponibilidade, configurar restrições, consultar os alunos associados, criar propostas de horário, confirmar essas propostas e gerir as aulas persistidas;
- o aluno pode escolher um professor, consultar o professor associado, registar a sua disponibilidade, definir a sua carga horária semanal pretendida e consultar as aulas que lhe estão atribuídas;
- a navegação da aplicação adapta-se ao perfil autenticado, apresentando ecrãs e operações distintas para cada perfil.

A gestão de sessão local foi integrada no frontend, permitindo preservar informação sobre o utilizador autenticado e suportar a adaptação dos fluxos da aplicação ao papel desempenhado. Esta distinção entre perfis foi também refletida no modelo de dados e na lógica dos ecrãs principais.

4.3 Gestão de Disponibilidades

A funcionalidade de gestão de disponibilidades representa uma parte fundamental da solução, pois constitui a base sobre a qual são construídas as propostas de horário. Tanto professores como alunos podem registar a sua disponibilidade semanal, indicando os dias e intervalos horários em que se encontram disponíveis.

Numa fase inicial da implementação, a disponibilidade foi modelada de forma mais simples, com um único intervalo associado a cada dia. Contudo, verificou-se que esta abordagem não representava adequadamente situações reais, em que um mesmo dia pode incluir múltiplos blocos horários separados. Por esse motivo, a funcionalidade foi evoluída para suportar múltiplos intervalos por dia, permitindo representar cenários como, por exemplo, disponibilidade durante a manhã e novamente durante a tarde no mesmo dia da semana.

Esta evolução teve impacto direto na interface e na lógica da aplicação, permitindo:

- acrescentar vários intervalos no mesmo dia;
- definir horas de início e fim específicas para cada bloco;
- remover intervalos de forma individual;

- persistir e recuperar corretamente essa informação.

A gestão de disponibilidades foi implementada de forma a suportar registo estruturado no frontend e persistência remota no backend, mantendo no dispositivo apenas mecanismos auxiliares de suporte local.

4.4 Gestão de Restrições

Para além das disponibilidades, o sistema implementa um conjunto de restrições que orientam a construção do horário do professor. Estas restrições representam regras de negócio relevantes para o problema e são definidas a partir do perfil do professor.

Na implementação atual, o professor pode configurar:

- número máximo de horas diárias;
- duração de cada sessão;
- número máximo de alunos por sessão;
- número máximo de sessões por aluno por dia.

A introdução desta funcionalidade permitiu passar de uma simples compatibilização de horários para um problema de planeamento mais próximo do caso de uso real. As restrições são persistidas e utilizadas posteriormente pelo mecanismo de criação de horários, desempenhando um papel central na validação das sessões propostas. Para além das restrições configuradas pelo professor, o sistema passou também a permitir ao aluno definir a sua carga horária semanal pretendida. Esta informação é integrada no algoritmo de criação de horários, evitando atribuições semanais acima do limite definido para cada estudante.

4.5 Associação entre Alunos e Professores

Outro aspeto relevante da implementação foi a definição do modelo de associação entre alunos e professores. A solução adotada considera que cada aluno pode escolher um único professor, enquanto um professor pode ter vários alunos associados.

Esta decisão foi refletida:

- no modelo de dados;
- na lógica do frontend;
- nos endpoints do backend;
- nos ecrãs de seleção e visualização de alunos.

4.6 Criação de Propostas de Horário

A criação de horários constitui o núcleo funcional do projeto. Na versão final, este processo evoluiu de um modelo puramente abstrato para uma abordagem em dois níveis: a criação de uma proposta semanal baseada em *templates* e a subsequente instanciação de aulas concretas e calendarizadas na base de dados com datas do ano civil.

A lógica de criação parte de uma representação em blocos temporais, obtidos a partir dos intervalos de disponibilidade registados. Estes blocos são analisados pelo algoritmo para identificar combinações viáveis entre o professor e os seus alunos. O processo de construção das propostas considera, de forma combinada, os seguintes aspetos:

- compatibilidade de disponibilidade entre professor e alunos;
- limite máximo de alunos por sessão;
- limite de sessões por aluno no mesmo dia;
- limite de carga diária do professor;
- restrições semanais de carga horária do aluno.

Após a proposta ser validada e aceite, o sistema procede à criação em lote das aulas reais. Para gerir o vínculo e a consistência destas sessões ao longo do tempo, introduziu-se o conceito de identificador de série (*seriesId*) e regras de recorrência (*RecurrenceType*). Uma aula pode ser criada como um evento isolado ou fazer parte de uma sequência repetitiva (tipicamente semanal), sendo replicada na base de dados com datas concretas de acordo com o número estipulado de ocorrências.

Esta distinção veio conferir ao sistema a flexibilidade necessária para o quotidiano letivo, permitindo que alterações pontuais ou cancelamentos de sessões individuais coexistam com a manutenção e integridade da série de horários planeada originalmente.

4.7 Visualização do Horário Criado

Após a criação das sessões, a aplicação apresenta o horário de forma organizada e legível. Em vez de mostrar apenas uma lista simples de resultados, foi construída uma visualização semanal com organização por dia, onde cada sessão aparece com o respetivo intervalo horário e a lista de alunos atribuídos.

Esta representação permite ao utilizador compreender facilmente:

- em que dias existem sessões;
- quais os intervalos temporais ocupados;

- que alunos foram alocados a cada sessão;
- em que dias não existem sessões previstas.

A implementação desta visualização constitui uma componente importante, pois transforma o resultado do algoritmo numa representação diretamente compreensível e utilizável pelo utilizador final. Para além da proposta semanal abstrata, a aplicação passou também a suportar a visualização de aulas concretas já persistidas, organizadas por semana e por intervalo temporal. Esta distinção entre proposta e aulas reais tornou-se importante para separar a fase de planeamento da fase operacional do sistema.

4.8 Gestão Dinâmica de Aulas e Presenças

Com o backend e as aplicações móveis totalmente integrados, foi possível expandir o sistema com um conjunto de ecrãs e fluxos operacionais destinados ao dia a dia do corpo docente e discente.

No lado do docente, através do *TeacherLessonsScreen*, passou a ser possível gerir o ciclo de vida das aulas. O professor dispõe de uma vista semanal organizada por datas concretas, onde pode criar novas aulas calendarizadas, editar o horário/data (com validação automática no backend para evitar sobreposição de horários do docente) e efetuar o cancelamento individual ou em série de aulas (o que cria os emails automáticos). É também neste ecrã que se faz a marcação e alteração de presenças dos alunos, cujos dados agregados alimentam um resumo estatístico consultável pelo docente.

Do lado do estudante, o ecrã *StudentLessonsScreen* oferece uma visão clara e cronológica do seu horário pessoal para a semana corrente. O aluno consegue verificar o estado de cada aula (confirmada ou cancelada) e acompanhar o estado das suas aulas e a informação de assiduidade registada no sistema.

4.9 Integração Progressiva com o Backend

Ao longo da implementação, foi também sendo reforçada a integração entre o frontend e o backend. Algumas operações começaram por ser validadas localmente, recorrendo à persistência em Room, e foram posteriormente adaptadas para consumir a API REST e interagir com o PostgreSQL.

Este processo de integração foi realizado de forma progressiva, permitindo validar cada funcionalidade antes da passagem completa para a componente remota. A implementação atual representa uma base funcional robusta, marcada por uma redução gradual da dependência de mecanismos locais auxiliares e pelo reforço do papel do backend como componente central do sistema.

Capítulo 5

Backend e API

O backend do sistema foi desenvolvido com o objetivo de centralizar a persistência remota de dados e disponibilizar uma API capaz de suportar os principais fluxos funcionais da aplicação. Esta componente desempenha um papel essencial na arquitetura da solução, funcionando como ponto de articulação entre a aplicação Android e a base de dados PostgreSQL.

A introdução do backend permitiu ultrapassar as limitações de uma solução exclusivamente local, abrindo caminho para uma estrutura mais próxima de um sistema real distribuído, onde os dados principais deixam de depender apenas do armazenamento existente no dispositivo do utilizador.

5.1 Escolha Tecnológica

A implementação do backend foi realizada em Kotlin, utilizando Ktor como framework principal para criação da API REST. Esta escolha revelou-se adequada ao contexto do projeto, sobretudo por permitir manter consistência com a linguagem já utilizada no frontend e por oferecer uma base relativamente simples e leve para a construção do servidor.

A persistência foi suportada por PostgreSQL, escolhido por ser um sistema de gestão de bases de dados relacional robusto, amplamente utilizado e adequado ao armazenamento estruturado das entidades do projeto. Para a interação com a base de dados foi utilizada a biblioteca Exposed, que facilita a definição de tabelas e a execução de operações de acesso a dados em Kotlin.

No conjunto, estas tecnologias permitiram construir uma base sólida, conciliando simplicidade de desenvolvimento com capacidade de evolução futura.

5.2 Estrutura do Backend

O backend foi organizado em vários componentes com responsabilidades distintas, de forma a manter o código compreensível e modular.

Entre os principais elementos da sua estrutura destacam-se:

- ficheiro principal de arranque da aplicação, responsável pela inicialização do servidor;
- configuração das rotas da API;
- modelos e DTOs utilizados na troca de informação entre cliente e servidor;
- serviços responsáveis por partes relevantes da lógica do sistema;
- configuração e inicialização da ligação à base de dados;
- definição das tabelas persistidas em PostgreSQL.

Esta organização permite separar as diferentes preocupações do sistema, distinguindo a exposição de endpoints, a representação dos dados, a lógica funcional e o acesso à persistência.

5.3 Modelo de Persistência no Backend

A camada de persistência do backend foi concebida para suportar as entidades centrais do domínio do projeto. O sistema focava-se no armazenamento de professores, alunos, associações entre ambos, disponibilidades, restrições dos professores e credenciais de autenticação. Para a versão final, este modelo foi expandido de forma a suportar o ciclo de vida completo das aulas e as novas regras de negócio do algoritmo, passando a incluir na sua totalidade:

- professores;
- alunos;
- associação entre alunos e professor;
- disponibilidades de horários;
- restrições associadas ao professor;
- restrições semanais do aluno;
- instâncias de aulas concretas calendarizadas;
- relação many-to-many entre alunos e aulas com controlo de presenças;
- credenciais necessárias à autenticação dos utilizadores.

A informação é persistida em PostgreSQL, permitindo manter um repositório central do estado do sistema. Esta abordagem representa um passo importante relativamente às fases iniciais do projeto, em que a aplicação dependia fortemente de persistência local (*Room*), que passou a ter um papel puramente auxiliar de cache. Com a consolidação do backend, a responsabilidade da gestão de dados principais ficou totalmente centralizada nesta componente remota, garantindo a integridade dos dados.

No caso das disponibilidades, o backend armazena os intervalos registados por professores e alunos, utilizando essa informação posteriormente para a criação de propostas de horário. Relativamente às restrições, o modelo foi atualizado para incluir não só as regras do professor, mas também o limite de horas semanais do aluno (*weeklyHours*), permitindo que o algoritmo de criação de horários processe estas condicionantes.

A maior evolução no modelo de dados prende-se com a transição de um modelo abstrato de horários para a persistência de dados reais. Através da *LessonTable*, o sistema consegue agora mapear cada aula a um dia e hora específicos do calendário, agrupando-as por séries de recorrência (*seriesId*). Complementarmente, a introdução da *LessonStudentTable* viabilizou o registo individualizado de assiduidade (*attendance*), permitindo extrair resumos estatísticos de presenças por aluno ou por turma.

No contexto da autenticação, manteve-se o uso de *hashing* de palavras-passe com recurso a BCrypt. Desta forma, as credenciais continuam a ser protegidas contra armazenamento em texto simples, reforçando a segurança da solução face às abordagens iniciais.

5.4 API REST Implementada

A API desenvolvida no backend foi desenhada para suportar os fluxos principais do sistema. Os endpoints existentes permitem realizar operações de registo, autenticação, consulta, associação de utilizadores, gestão de disponibilidades, gestão de restrições, criação de propostas de horário, persistência de aulas concretas, marcação de presenças, consulta de histórico, cancelamento e remarcação de aulas, bem como envio de notificações.

Entre os principais endpoints implementados incluem-se:

- verificação de disponibilidade do servidor;
- criação e consulta de professores;
- criação e consulta de alunos;
- autenticação de utilizadores;
- associação e desassociação de alunos a professores;
- consulta dos alunos de um professor;
- criação, remoção e consulta de disponibilidades;
- criação e consulta de restrições do professor e do aluno;
- criação de propostas de horário;
- geração e persistência de aulas concretas com datas reais;

- consulta semanal e histórica de aulas;
- marcação de presenças e obtenção de resumos de assiduidade;
- cancelamento individual e cancelamento em série;
- remarcação de aulas;
- envio de notificações por email aos alunos.

A utilização de uma API REST permitiu estabelecer um contrato claro entre o frontend e o backend, facilitando a integração progressiva da aplicação Android com a componente remota. O formato JSON foi adotado como mecanismo de serialização dos dados, permitindo representar os pedidos e respostas de forma simples e consistente. Durante a evolução do projeto, foi também introduzido um endpoint específico de autenticação, permitindo validar no backend as credenciais dos utilizadores. Esta alteração representou um passo importante na arquitetura da solução, uma vez que a validação do login deixou de depender da comparação local de dados no frontend, passando a ser tratada de forma centralizada na API.

5.5 Criação de Horários no Backend

Uma das evoluções mais relevantes foi a passagem da lógica de criação de horários para o backend. Esta decisão permite centralizar a lógica principal do sistema e reduzir a duplicação de comportamento entre cliente e servidor.

A partir da informação armazenada, o backend:

- obtém a disponibilidade do professor;
- obtém as disponibilidades dos alunos associados;
- carrega as restrições definidas para o professor;
- carrega o limite semanal de horas definido para cada aluno;
- divide os intervalos em blocos temporais adequados;
- cria sessões compatíveis com os dados disponíveis;
- persiste essas sessões como aulas concretas com datas reais e recorrência configurável.

Esta lógica permite que a proposta de horário seja construída de forma coerente e reutilizável, garantindo que diferentes clientes possam consumir o mesmo comportamento central sem necessidade de replicar o algoritmo localmente. Com esta evolução, o backend deixou de produzir apenas um resultado abstrato de planeamento, passando também a gerir o ciclo de vida das aulas geradas, incluindo a sua persistência, consulta, alteração, cancelamento e ligação a séries de recorrência.

5.6 Serviço de Notificações por Email

De modo a assegurar uma comunicação fluida e automatizada entre o sistema e os utilizadores, foi desenvolvido e integrado o *EmailService*. Este serviço assenta na biblioteca *Jakarta Mail* / *JavaMail* e comunica com o servidor *SMTP do Gmail* para o envio de alertas em tempo real.

De forma a mitigar dependências externas em ambiente de desenvolvimento e testes, implementou-se um mecanismo de simulação. Caso as variáveis de ambiente necessárias para a ligação SMTP não estejam configuradas, o serviço reverte automaticamente para um modo de simulação, escrevendo o conteúdo e o destino dos emails diretamente nos *logs* da aplicação.

Atualmente, o ecossistema do backend despoleta notificações automáticas por email em três cenários críticos:

- Cancelamento de Aula Individual: Notifica todos os alunos inscritos numa determinada aula de que a respetiva sessão foi cancelada.
- Cancelamento de Série: Quando o professor cancela uma recorrência (uma série de aulas), é enviado um aviso geral a todos os alunos.
- Remarcação de Aula: Notifica os estudantes sobre alterações na data, hora ou moldes de uma aula já agendada, após a validação de conflitos.

Para além destes cenários automáticos, o sistema suporta também o envio manual de avisos livres pelo professor para todos os alunos associados ou apenas para um subconjunto selecionado, permitindo utilizar a infraestrutura de email como mecanismo de comunicação operacional.

5.7 Validação com Postman

A validação do backend foi realizada com recurso ao Postman, ferramenta utilizada para testar os endpoints desenvolvidos antes da integração total com o frontend. Este processo foi importante para confirmar que os contratos da API estavam corretos e que as operações principais produziam os resultados esperados.

Durante os testes foram validados fluxos como:

- criação de professores e alunos;
- autenticação de utilizadores;
- consulta de listas de utilizadores;
- associação e desassociação entre aluno e professor;

- registo, consulta e remoção de disponibilidades;
- registo e consulta de restrições do professor e do aluno;
- criação de propostas de horário;
- geração de aulas concretas com recorrência;
- consulta semanal e histórica de aulas;
- marcação de presenças;
- cancelamento individual e em série;
- remarcação de aulas;
- envio de notificações por email.

A utilização do Postman permitiu isolar o backend durante o processo de validação, facilitando a identificação de problemas antes da integração direta com a aplicação Android. Este passo revelou-se particularmente útil para testar a coerência das respostas devolvidas pela API e a persistência correta dos dados em PostgreSQL.

5.8 Integração com a Aplicação Android

Após a validação inicial dos endpoints, a integração com o frontend foi sendo realizada de forma progressiva. A aplicação Android passou a consumir a API para operações relacionadas com utilizadores, autenticação, disponibilidades, restrições, criação de horários, gestão de aulas persistidas, marcação de presenças, notificações e integração com Google Calendar.

Esta integração permitiu aproximar a solução da sua arquitetura final, em que o backend assume o papel de componente central na gestão dos dados e da lógica principal do sistema. Apesar de ainda existirem aspetos híbridos entre persistência local e remota, a evolução observada nesta fase demonstra que a base da comunicação cliente-servidor já se encontra estabelecida e funcional.

5.9 Síntese do Estado Atual do Backend

O backend já se encontra suficientemente estruturado para suportar os principais fluxos do projeto. A API implementada encontra-se funcional, a ligação à base de dados PostgreSQL foi estabelecida com sucesso e os principais endpoints, incluindo os de autenticação e criação de horários, já foram validados de forma independente e em conjunto com a aplicação móvel.

Este estado representa um marco importante no desenvolvimento da solução, pois demonstra que o projeto já não depende apenas de lógica local no frontend. O backend passou

a constituir uma parte ativa e central do sistema, permitindo sustentar o funcionamento atual da solução e a sua evolução futura para novas funcionalidades e cenários de utilização.

Capítulo 6

Testes e Validação

A validação do sistema foi realizada de forma progressiva ao longo do desenvolvimento, acompanhando a implementação das diferentes funcionalidades da aplicação e do backend. Esta abordagem permitiu testar isoladamente componentes específicos e, posteriormente, verificar o comportamento integrado da solução. Numa fase inicial, os testes incidiram sobretudo sobre a lógica de tratamento de disponibilidades, a criação de horários, o funcionamento da API e os fluxos principais da aplicação Android.

Na versão final, a estratégia de testes evoluiu de verificações maioritariamente manuais para a implementação de uma infraestrutura robusta de testes integrados e unitários no backend, garantindo a integridade das regras de negócio, dos endpoints da API e dos fluxos das aplicações móveis.

6.1 Infraestrutura e Testes Automatizados no Backend

Para assegurar a fiabilidade da lógica de negócio sem depender da base de dados de produção (PostgreSQL), foi montado um ecossistema de testes automatizados suportado por uma base de dados *H2* em memória partilhada.

Através da classe *TestDatabase*, implementou-se um mecanismo que executa um *reset* integral ao esquema de tabelas antes da execução de cada caso de teste. Esta abordagem garante o isolamento completo dos testes, permitindo que corram de forma rápida e previsível.

Desta forma, os testes automatizados cobrem atualmente os seguintes componentes lógicos e de serviço:

- **Processador de intervalos temporais:** Responsável por dividir disponibilidades em blocos utilizáveis pelo algoritmo. Os testes unitários permitem verificar aspetos como: o número de blocos criados para diferentes intervalos, a preservação correta do dia da semana, o comportamento em intervalos inválidos ou demasiado curtos, e o suporte a diferentes durações de bloco.
- **Mecanismo e algoritmo de criação de horários:** Responsável por construir as

propostas de sessões. Os testes unitários e de serviço (*ScheduleServiceTest*) validam a atribuição correta de alunos disponíveis, o respeito pelo número máximo de participantes por sessão, o respeito pelo número máximo de sessões por aluno por dia, as restrições semanais de carga horária do aluno (*weeklyHours*) e o comportamento em cenários sem alunos ou sem disponibilidade do professor.

- **Gestão de Aulas Concretas (*LessonServiceTest*):** Uma adição crítica da versão final que testa a lógica de persistência de aulas recorrentes com datas reais, a consulta de histórico, a marcação de presenças (*attendance*), os fluxos de cancelamento de séries (com simulação de notificações) e a detecção automática de conflitos em edições de horário.
- **Rotas e Contratos da API (*Integration Level*):** Recorrendo à funcionalidade *testApplication* do *Ktor*, foram criados testes de integração automatizados (*TeacherRoutesTest*, *StudentRoutesTest*, *AvailabilityRoutesTest*, *RestrictionsRoutesTest*, *ScheduleRoutesTest*, *LessonRoutesTest*, *AccountRoutesTest* e *AuthRoutesExtraTest*) que simulam pedidos HTTP contra o servidor e validam os payloads JSON e os códigos de resposta face ao esperado.

Estes testes constituem uma base importante de confiança para o comportamento do sistema, sobretudo nas partes em que a lógica é mais sensível a regras e combinações de dados.

6.2 Validação do Backend com Postman

Com a introdução dos testes automatizados via *Ktor* e *H2*, a ferramenta Postman deixou de ser a única forma de testar a API, transitando para um papel de validação manual durante o desenvolvimento concorrente das interfaces móveis.

Esta ferramenta revelou-se importante para validar em tempo real o comportamento físico do *PostgreSQL* e inspecionar os payloads de rotas mais complexas antes da sua integração final na aplicação. Os testes realizados no Postman integraram os fluxos originais e as novas rotas implementadas, cobrindo os seguintes cenários do domínio:

- verificação de disponibilidade do servidor (*/health*);
- criação de professores e alunos;
- autenticação de utilizadores e validação de credenciais (*/login*);
- consulta de listagens globais de professores e alunos;
- associação de alunos a professores e remoção desse vínculo;

- consulta de alunos associados a um professor;
- registo, consulta e remoção de lote de disponibilidades;
- registo, consulta e atualização de restrições do professor;
- criação e execução do algoritmo de criação de horários (*POST /lessons/generate*);
- consulta de horários semanais e históricos específicos de professores e alunos;
- alteração dinâmica do estado de presença de um aluno;
- remoção de lote de séries de aulas por *seriesId* e cancelamentos individuais;
- envio de alertas livres por e-mail aos alunos via *EmailService*.

Através destes testes foi possível confirmar que os métodos HTTP e os principais fluxos do domínio estavam corretamente representados no backend. Esta validação em duplicado permitiu também identificar e corrigir inconsistências entre o modelo de dados do frontend e o do backend, contribuindo para uma integração mais robusta entre os dois componentes do ecossistema.

6.3 Testes Funcionais da Aplicação Android

Ao nível da aplicação Android, os testes realizados incidiram sobre os principais fluxos funcionais disponíveis para cada tipo de utilizador. O objetivo foi validar o comportamento da interface, a navegação entre ecrãs, a persistência de dados e a integração progressiva com a API.

Entre os fluxos testados destacam-se:

- autenticação e gestão de sessão;
- distinção de comportamento entre professor e aluno;
- seleção e troca de professor por parte do aluno;
- visualização de alunos associados ao professor;
- definição de disponibilidades, incluindo múltiplos intervalos por dia;
- definição de restrições;
- criação e apresentação do horário;
- aceitação do horário e integração com Google Calendar.
- consulta de aulas persistidas e histórico;

- marcação de presenças por parte do professor;
- remarcação e cancelamento de aulas;
- envio manual de notificações aos alunos.

A validação funcional da aplicação foi importante para garantir que a interface suportava corretamente os comportamentos previstos pelo domínio do problema e que a informação apresentada ao utilizador era consistente com o estado armazenado localmente e, quando aplicável, com a informação obtida do backend. Foi também validada a integração com o Google Calendar no contexto do professor. Após a aceitação de uma proposta de horário, verificou-se que as sessões podiam ser registadas com sucesso no calendário do utilizador autenticado, confirmando o funcionamento do fluxo de autenticação Google e da utilização da respetiva API.

6.4 Validação da Criação de Horários

A criação de horários foi alvo de validação específica, dado tratar-se da funcionalidade central do projeto. Para esse efeito, foram construídos cenários de teste com diferentes combinações de disponibilidades, alunos e restrições, permitindo observar o comportamento do sistema em situações representativas.

Os testes incidiram sobre aspetos como:

- divisão correta dos intervalos de disponibilidade em blocos temporais;
- criação de sessões compatíveis com a disponibilidade do professor;
- alocação de alunos apenas a sessões em que se encontram disponíveis;
- respeito pelos limites de participantes por sessão;
- respeito pelo número máximo de sessões por aluno no mesmo dia;
- distribuição de alunos por diferentes blocos quando aplicável;
- respeito pelo limite semanal de horas definido para cada aluno.

Através destes testes foi possível confirmar que o projeto já produz propostas de horário coerentes com as regras definidas e com a informação introduzida no sistema. Embora a abordagem atual ainda possa ser melhorada, os resultados obtidos mostram que a funcionalidade principal do projeto já se encontra operacional.

6.5 Integração e Validação Progressiva

Um aspecto relevante da fase de testes foi a validação progressiva da integração entre frontend e backend. Em vez de tentar validar toda a solução apenas no final, as funcionalidades foram sendo testadas à medida que a comunicação remota era introduzida. Esta estratégia permitiu identificar mais cedo inconsistências em endpoints, modelos de dados, fluxos de autenticação e mecanismos de sincronização.

A coexistência de persistência local e persistência remota exigiu também atenção adicional, uma vez que parte da validação teve de assegurar que os dados obtidos da API se mantinham consistentes com o estado visível na aplicação. Esta abordagem incremental revelou-se útil para consolidar a solução final e estabelecer uma base sólida para futuras evoluções funcionais do sistema.

6.6 Síntese dos Resultados Obtidos

Os testes realizados permitiram concluir que o sistema apresenta um nível de funcionalidade significativo. O backend está operacional, os principais endpoints foram validados com sucesso, a aplicação Android executa os fluxos centrais do sistema, a autenticação encontra-se funcional e a criação de horários produz resultados coerentes em cenários representativos.

Apesar de persistirem aspetos a melhorar, nomeadamente ao nível da integração total entre componentes, da evolução futura do algoritmo e do polimento de algumas componentes da interface, a validação efetuada demonstra que a solução já consegue suportar os casos de uso mais importantes do projeto. Este resultado constitui um indicador positivo da robustez da solução atualmente desenvolvida e da sua capacidade de suportar futuras evoluções.

Capítulo 7

Estudo Exploratório com Large Language Models (LLMs) para Interpretação de Disponibilidades

Foi desenvolvido um estudo exploratório com o objetivo de avaliar a viabilidade de utilização de modelos locais e multimodais para interpretar disponibilidades de professores e alunos a partir de diferentes tipos de input. A motivação para este estudo surgiu da possibilidade de, no futuro, permitir à aplicação receber informação em formatos menos estruturados, como texto livre, imagens, *screenshots*, fotografias manuscritas ou áudio, transformando esses dados em informação útil para o sistema.

A ideia central consistiu em analisar se seria possível utilizar estes modelos como apoio ao registo de disponibilidades, reduzindo a necessidade de inserção exclusivamente manual e estruturada por parte do utilizador.

Até ao momento, o estudo foi dividido em três frentes principais:

- extração de texto a partir de imagens;
- interpretação de disponibilidades em texto e imagem;
- preparação da pipeline necessária para futura análise de áudio.

7.1 Modelos e Ferramentas Avaliados

Durante esta fase foram considerados diferentes modelos e ferramentas, com papéis distintos no processamento dos dados:

- *LLaVA* [1], utilizado numa fase inicial como modelo multimodal geral;
- *glm-ocr:q8_0*, avaliado como solução local dedicada a OCR;
- *qwen2.5vl:3b* [2], utilizado como modelo multimodal para interpretação de imagem e texto;

- *ffmpeg*, instalado como ferramenta de apoio à preparação de ficheiros áudio;
- *Whisper* [3], considerado para uma futura etapa de transcrição automática de áudio.

Entre os modelos testados, o *llava* teve um papel mais exploratório e não foi mantido como solução principal. O *glm-ocr:q8_0* foi inicialmente escolhido por ser leve, local e vocacionado para OCR. Já o *qwen2.5vl:3b* destacou-se como o candidato mais promissor para interpretação de inputs visuais e textuais.

7.2 Resultados Obtidos com OCR e Multimodalidade

Os testes experimentais focaram-se na capacidade de extração de dados e na robustez de estruturação de ambos os modelos perante diferentes níveis de ruído no input.

7.2.1 Avaliação do GLM-OCR (q8_0)

O modelo demonstrou eficácia restrita a cenários controlados, exibindo as seguintes características:

- **Pontos Fortes:** Desempenho positivo na leitura de texto digital limpo, tabelas estruturadas e *screenshots*.
- **Limitações Semântica:** Incapacidade de estruturar dados em formatos estritos (como JSON válido) e tendência para traduzir os dias da semana para inglês.
- **Falhas em Manuscritos:** Elevada taxa de erro em fotografias de escrita manual, introduzindo linhas inexistentes e alterando horários originais.

Concluiu-se que o *GLM-OCR* serve apenas como ferramenta de OCR simples, sendo inviável para a ingestão autónoma do sistema.

7.2.2 Avaliação do Qwen2.5-VL (3b)

O *Qwen2.5-VL* apresentou um desempenho marcadamente superior, validando a abordagem *vision-language* em cenários reais:

- **Robustez Contextual:** Elevada estabilidade em registos manuscritos, preservando os intervalos temporais factuais sem inventar informação, apesar de pequenos erros ortográficos isolados.
- **Versatilidade de Inputs:** Extração bem-sucedida a partir de texto livre, imagens e *screenshots*.
- **Interpretação Semântica:** Capacidade de compreender disponibilidades descritas em linguagem natural, mapear preferências e isolar indisponibilidades.

Face aos resultados obtidos, o *Qwen2.5-VL (3b)* fixou-se como a solução ideal para suportar a futura componente de interpretação inteligente de disponibilidades.

7.3 Estado Atual da Componente de Áudio

A componente de áudio ainda não foi validada de forma completa nesta fase do projeto. Foi preparado um ficheiro de teste e instalada a ferramenta *ffmpeg* para apoio ao tratamento deste tipo de input. No entanto, a etapa de transcrição automática ainda não foi concluída, mantendo-se o *Whisper* como a solução mais provável para evolução futura.

Dado que o *qwen2.5vl:3b* não interpreta áudio diretamente, a arquitetura prevista e pensada para este caso passa por uma pipeline em duas etapas:

- transcrição do áudio para texto com um modelo de reconhecimento de fala;
- interpretação desse texto com o modelo multimodal selecionado.

Assim, a abordagem atualmente considerada para este tipo de input corresponde a:

$$\text{audio} \rightarrow \text{Whisper} \rightarrow \text{texto transcrito} \rightarrow \text{qwen2.5vl:3b}$$

7.4 Proposta de Integração no Projeto

Embora esta componente não tenha sido integrada na solução final, o estudo realizado permite já delinear uma proposta de arquitetura para evolução futura do sistema.

A solução mais promissora identificada até ao momento assenta nos seguintes fluxos:

- texto direto $\rightarrow$ *qwen2.5vl:3b*;
- imagem simples $\rightarrow$ *qwen2.5vl:3b*;
- *screenshot* $\rightarrow$ *qwen2.5vl:3b*;
- fotografia manuscrita $\rightarrow$ *qwen2.5vl:3b* com validação humana;
- áudio $\rightarrow$ *Whisper* + *qwen2.5vl:3b*.

Esta abordagem permitiria acrescentar à aplicação uma forma mais flexível de recolha de disponibilidades, especialmente útil em contextos em que o utilizador dispõe da informação em formatos menos estruturados. Ao mesmo tempo, os resultados obtidos mostram que esta integração deverá ser feita com cuidado, prevendo sempre mecanismos de validação e confirmação por parte do utilizador, sobretudo em casos de manuscrito ou áudio.

7.5 Síntese do Estudo

O estudo realizado permitiu concluir que, até ao momento, não foi identificado um único modelo capaz de tratar de forma totalmente fiável todos os tipos de input considerados. Ainda assim, foi possível identificar uma abordagem tecnicamente promissora para evolução futura da solução, com destaque para o *qwen2.5vl:3b* como principal candidato para imagem e texto, e para o *Whisper* como etapa complementar no caso do áudio.

Capítulo 8

Estado Final do Sistema e Avaliação

A solução já não se encontra limitada a um protótipo exclusivamente local. Com a introdução do backend em *Ktor*, a integração com *PostgreSQL* e a utilização de uma API REST funcional, o sistema passou a suportar um cenário de utilização multiutilizador próximo de um contexto real.

Este capítulo apresenta uma avaliação crítica do sistema edificado, confrontando os avanços alcançados face às limitações previamente identificadas e listando, de forma transparente, os requisitos que transitam como trabalho futuro.

8.1 Funcionalidades Concluídas e Avaliação de Progresso

O progresso e as metas atingidas são avaliados nos seguintes vetores:

- **Consolidação da Persistência e Fim do Modelo Híbrido:** Uma das maiores limitações da fase anterior prendia-se com a dependência de fluxos híbridos e a falta de sincronização clara entre o *Room* (local) e o *PostgreSQL* (remoto). Na versão final, esta fragilidade foi significativamente reduzida. A persistência local passou a assumir um papel essencialmente auxiliar de *cache* e suporte à interface, enquanto a centralização dos dados principais ficou cometida ao backend.
- **Melhoria do Algoritmo de Horários:** O mecanismo de criação de propostas, que antes operava em cenários muito simplificados, foi evoluído para lidar com condicionantes mais complexas do domínio, destacando-se a integração bem-sucedida do limite de carga horária semanal individualizada de cada aluno.
- **Criação Concreta e Ciclo de Vida de Aulas:** O sistema passou a transpor com sucesso as propostas semanais abstratas para instâncias físicas de aulas agendadas no ano civil, permitindo ao corpo docente o controlo de aulas, faltas e presenças com datas reais persistidas na base de dados.

- **Automatização de Notificações:** A integração do *EmailService* colmatou a necessidade de comunicação dinâmica, garantindo que cancelamentos e remarcações disparados via aplicação móvel criam alertas SMTP automáticos para os visados.
- **Alargamento Crítico da Cobertura de Testes:** A validação progrediu de um carácter essencialmente manual com Postman para uma automatização alargada, tirando partido de testes de integração de rotas e serviços baseados numa base de dados *H2* em memória.

8.2 Limitações Reais e Trabalho Futuro

Apesar de a solução cumprir com rigor o propósito focado na gestão e planeamento de horários entre explicadores e alunos, a gestão do tempo e a priorização da estabilidade da API ditaram o adiamento de algumas funcionalidades idealizadas. Assumem-se como limitações reais do projeto final:

- **Gestão e Alocação de Salas:** O algoritmo e a base de dados detetam sobreposições horárias de utilizadores, mas o sistema não contempla a componente física de inventário e alocação de salas de aula ou espaços de tutorias.
- **Múltiplas Formas de Input de Disponibilidades:** Embora o estudo exploratório com o modelo multimodal *qwen2.5vl:3b* e com o *Whisper* tenha delineado uma *pipeline* tecnicamente promissora, a interpretação automatizada de disponibilidades a partir de texto livre, imagem ou áudio não foi integrada na aplicação final. A recolha de dados continua a depender exclusivamente da inserção manual e estruturada na interface móvel.

Estas limitações não comprometem o valor prático da solução obtida, mas demonstram um caminho claro e fundamentado para futuras versões do produto.

Capítulo 9

Conclusões

O desenvolvimento do projeto *Sistema de Planeamento de Horários* permitiu demonstrar a viabilidade técnica da solução proposta e validar com sucesso os seus principais fluxos funcionais. Ao longo do projeto foi possível consolidar uma arquitetura robusta composta por aplicação móvel, backend e persistência remota, inserindo a solução num contexto de utilização real. A solução final entregue suporta de forma consistente as operações centrais planeadas, como o registo e autenticação de utilizadores, a associação entre alunos e professores, o registo de disponibilidades, a definição de restrições complexas e a construção automatizada de propostas de horário. A integração bem-sucedida da aceitação do horário com o *Google Calendar* do professor acrescentou um valor prático imediato à solução, reforçando a sua utilidade no quotidiano de um centro de estudos. Para além da implementação funcional, o projeto permitiu validar com rigor a comunicação síncrona entre frontend e backend, otimizar a usabilidade da interface móvel e consolidar mecanismos críticos de segurança, incluindo a autenticação remota e o armazenamento seguro de credenciais. Os testes automatizados realizados demonstraram que a solução responde de forma coerente e estável aos principais casos de uso identificados. O trabalho desenvolvido permitiu ainda abrir uma linha exploratória complementar inovadora relacionada com a interpretação de disponibilidades a partir de inputs não estruturados, com recurso a modelos multimodais de Inteligência Artificial. Embora esta componente não tenha sido integrada na aplicação final devido à priorização da estabilidade do sistema base, o estudo exploratório efetuado permitiu identificar abordagens robustas e documentadas para o tratamento de texto, imagem e áudio em futuras evoluções do sistema. Em síntese, o produto final demonstra utilidade prática real, coerência arquitetural e cumpre os objetivos pedagógicos e técnicos centrais definidos para este projeto.

Referências

- [1] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023.
- [2] Qwen Team. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [3] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.
- [4] JetBrains. Kotlin documentation, 2026. [Online; acedido em 1-Junho-2026].
- [5] Android Developers. Guide to app architecture, 2026. [Online; acedido em 1-Junho-2026].
- [6] Android Developers. Jetpack compose documentation, 2026. [Online; acedido em 1-Junho-2026].
- [7] Android Developers. Save data in a local database using room, 2026. [Online; acedido em 1-Junho-2026].
- [8] JetBrains. Ktor documentation, 2026. [Online; acedido em 1-Junho-2026].
- [9] PostgreSQL Global Development Group. Postgresql documentation, 2026. [Online; acedido em 1-Junho-2026].
- [10] Google for Developers. Google calendar api overview, 2026. [Online; acedido em 1-Junho-2026].
- [11] Niels Provos and David Mazières. A future-adaptable password scheme. In *1999 USENIX Annual Technical Conference*. USENIX Association, 1999.
- [12] Asimov Academy. Llms multimodais, 2024. [Online; acedido em 1-Junho-2026].